E-graphs modulo theories (extended abstract)

Sofia Brookie

April 18, 2026

1 Introduction

E-graphs struggle with a combinatorial explosion when reasoning about certain theories. A notorious case is that of AC theories, those containing associative and commutative operators: the space complexity of equality saturation is doubly-exponential in the size of the input in the worst case.

Can equality saturation for AC theories be done in a more efficient way? There have been several proposals for extending E-graphs to *E-graphs modulo theories* ([8], [4], [5]). We want to combine *theory-specific* reasoning methods for AC together with the general mechanisms of E-graphs and equality saturation in order to efficiently deal with AC theories. This approach is named after and inspired by the success of theory-specific methods in other automated reasoning tasks, such as in *Satisfiability Modulo Theories* solvers (SMT solvers). The merits of a theory-specific approach in automated reasoning are advocated for in Shankar [6].

1.1 An example

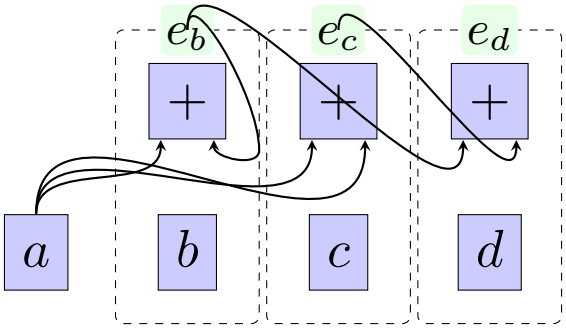
The natural first step is to simply not add any different representations of terms that are equal up to AC; thus, nodes in the E-graph for AC operators will be variadic and we will consider their children unordered. This is the key idea from prior work ([8], [4], [5]).

We will show that something extra is needed. Consider the equations

$$a + b = b \tag{1}$$
$$a + a = c \tag{2}$$
$$c + b = d \tag{3}$$

taken modulo associativity of $+$.

We can represent them with an E-graph as follows:



Consider the equation $b = d$, which follows from the simple deduction

$$b = a + b = a + (a + b) = (a + a) + b = c + b = d$$

However, e_b and e_d are two distinct E-classes in the E-graph! Why not?

The central step in the deduction relies on two different associations of the term $a + a + b$. The association $(a + a) + b$ is correctly assigned to class e_d, while the association $a + (a + b)$ is correctly assigned to the class e_b. However, given that we want to work modulo theories, we need to discover the fact that $e_b = e_d$, or we are throwing out some of the consequences derivable from the theory.

1.2 How E-graphs handle ordinary critical pairs, but not AC-critical pairs

To get a better perspective on this problem, we use ideas from rewriting and in particular the notion of Abstract Congruence Closure ([1]). We can think of E-graphs as presenting a rewriting system, and the above E-graph as containing the rewrite rules

$$\begin{aligned} b &\to e_b \\ c &\to e_c \\ d &\to e_d \\ a + e_b &\to e_b \\ a + a &\to e_c \\ e_c + e_b &\to e_d \end{aligned}$$

(A distinguished equivalence class for a is omitted to make the diagram easier to follow)

A critical pair for a rewriting system is a term that can be rewritten two different ways[1]. The perspective of Abstract Congruence Closure is that congruence closure, the core algorithm for 'rebuilding' E-graphs, constructs a *confluent term rewriting system* on an extended signature. Handling critical pairs is required for confluence, which ensures that a term rewrites into a single result; if we have a term t that rewrites to both e_a and e_b, we ought to automatically *deduce* that the E-classes e_a and e_b should be merged.

By introducing new constants $e_1, e_2, \ldots$ as E-ids, we can 'flatten' an input term $f(\ldots, g(h(\ldots)), \ldots)$ into just $f(\ldots, e_i, \ldots)$ and then assign it the name e_k for some k.

Now, to ensure the convergence of the rewrite system, we can look at two kinds of cases where a term could rewrite in two ways:

$e_1 \leftarrow f(\ldots e_i \ldots) \rightarrow e_2$ in which case, we ought to deduce that $e_1 = e_2$. This corresponds to merging E-classes which would contain duplicates of the same E-node.

$f(\ldots e_i \ldots) \rightarrow e_1$ and $e_i \rightarrow e_2$, in which case we ought to deduce that $f(\ldots e_2 \ldots) = e_1$. This corresponds to canonizing the E-nodes using the union-find structure.

By fixpoint propagation, resolving these two cases are enough to resolve all cases of critical pairs with overlaps on more deeply nested subterms.

E-graph rebuilding can handle ordinary critical pairs, but this doesn't cover our above example, which is an *AC-critical pair*.

1.3 Equality satuation: handling AC-critical pairs by brute force

E-graphs (not modulo theories) handle AC-critical pairs the same way as critical pairs for any other theory, with equality saturation: we *ground* the AC identities by matching either side with the E-graph, and adding the equivalent other side (if it is not present) and merging the equivalence classes corresponding to either side.

Why do E-graphs fare poorly with AC? The two identities, (A) $\forall xyz.\ x + (y + z) = (x + y) + z$ and (C) $\forall xy.\ x + y = y + x$, can be semantically characterized as saying that two AC-terms are equal if they are equal as non-empty multisets of their subterms. Equality saturation will handle these identities by eventually deriving that an input term $a_1 + a_2 + \ldots + a_n$, thought of as the multiset $\{a_1, a_2, \ldots, a_n\}$, is in the same E-class as every E-node of the form $s + t$ where s and t partition the multiset $\{a_1, a_2, \ldots, a_n\}$. There are $2^n - 1$ non-empty

[1] It is also required to be minimal among such terms, but we gloss over this here

multisets s and t which we can partition a starting multiset of size n into[2], and one E-class will need to be generated for each. So, to reach saturation, we must create $O(2^n)$ E-classes, and it is easy to see that each E-class will, on average, have its own $O(2^n)$ E-nodes to represent every multiset partition of the multiset it represents.

These 'redundant' E-nodes and E-classes play a role: any of the $O(2^{2^n})$ AC-subterms of the input could be part of an AC-critical pair, and so equality saturation eagerly assigns new E-classes to them, just in case. Introducing these extra E-classes and E-nodes is seemingly the best we can do in general unless we have a clever way to detect which ground instances of the identities are needed in order to ensure confluence modulo those identities.

In our above example, the associative identity would match against $a + (a + b)$, which is a term represented in our E-graph, and then merge this term (assigned to equivalence class e_b) with the corresponding other side of the associativity identity (assigned to equivalence class e_d *if* we have also added the instance of commutativity $c + b = b + c$ to the graph. Otherwise, adding this instance of commutativity later on and then rebuilding will correctly find the need to merge these classes).

1.4 EMTs: Handling AC-critical pairs less naïvely

We don't need to add all AC-subterms to find critical pairs. It suffices to consider every pair of E-nodes with an AC symbol and check them for 'overlap'. In our example, the E-nodes $a + e_b$ and $a + a$ form a potential critical pair $a + a + e_b$ as they overlap on the subterm a. We see that this implies the equation $a + e_b = e_c + e_b$, which can be added to the E-graph, which will merge the classes e_b of the LHS and e_d of the RHS.

This procedure always terminates ([1]). It is, in fact, a special case of Buchberger's algorithm for finding Gröbner bases of sets of polynomials ([7]), a well-studied algorithm with a lot of work done in optimizing it to be fast enough in practice.

2 Towards EMTs

Seeing that resolving T-critical pairs is a key ingredient for E-graphs modulo T, we look to see if there are better ways to do this for certain Ts (like AC) beyond the naïve, general approach of equality saturation.

We can show that this is equivalent to a known problem, the (uniform) word problem over T.

[2]If the multiset has an element of multiplicity more than 1, some of these will be the same, so this number will be reduced. However, we consider the worst case

For AC, the word problem is known to be ESPACE-complete ([2]). This provides a theoretical lower bound on how much EMTs for AC can hope to achieve. Nonetheless, this is better than equality saturation for AC which takes doubly-exponential space, putting it in the class EESPACE.

We can hope that AC will be solvable 'fast enough in practice', as is typically the case for many problems such as SAT-solving. The connection with the well-studied Buchberger's algorithm makes this reasonable to hope for.

For A, the word problem is known to be undecidable ([3]), so the best we can hope to achieve in general is provide an incremental method to find and resolve T-critical pairs, with no guarantee that this process will ever end. Nonetheless, working modulo A can improve on the expected cubic space overhead of the intermediate E-graphs created by equality saturation with A.

For certain theories, there are nicer bounds: Abelian groups, aka ACUN, have a word problem solvable in polynomial time by computing Hermite normal form. For vector spaces, even faster solving via Gaussian elimination is possible.

2.1 Results

We are currently implementing an EMT system that uses Buchberger's algorithm for AC. As we have argued, we need something like this to make an EMT system that

References

[1] Leo Bachmair and Ashish Tiwari. "Abstract Congruence Closure and Specializations". In: *Automated Deduction - CADE-17*. Berlin, Heidelberg, 2000, pp. 64–78. DOI: 10.1007/10721959_5.

[2] Ernst W Mayr and Albert R Meyer. "The complexity of the word problems for commutative semigroups and polynomial ideals". In: *Advances in Mathematics* 46.3 (Dec. 1, 1982), pp. 305–329. ISSN: 0001-8708. DOI: 10.1016/0001-8708(82)90048-2. URL: https://www.sciencedirect.com/science/article/pii/0001870882900482.

[3] Carl-Fredrik Nyberg-Brodda. *G. S. Tseytin's seven-relation semigroup with undecidable word problem*. Jan. 22, 2024. DOI: 10.48550/arXiv.2401.11757. arXiv: 2401.11757[math]. URL: http://arxiv.org/abs/2401.11757.

[4] Pavel Panchekha. *LIN-graphs, an Egraph modulo T*. Pavel's Blog. May 23, 2025. URL: https://pavpanchekha.com/blog/lin-graphs.html.

[5] Saul Shanabrook. *Custom Data Structures in E-Graphs*. PLSE Blog. Feb. 24, 2026. URL: https://uwplse.org/2026/02/24/egglog-containers.html.

[6] Natarajan Shankar. "Little Engines of Proof". In: *FME 2002:Formal Methods—Getting IT Right*. Vol. 2391. Berlin, Heidelberg, 2002, pp. 1–20. ISBN: 978-3-540-43928-8 978-3-540-45614-8. DOI: 10.1007/3-540-45614-7_1. URL: http://link.springer.com/10.1007/3-540-45614-7_1.

[7] Ashish Tiwari. *Decision procedures in automated deduction*. State University of New York at Stony Brook, 2000. URL: https://search.proquest.com/openview/30046982652b13cadc7b85d653cdf547/1?pq-origsite=gscholar&cbl=18750&diss=y.

[8] Philip Zucker. *Omelets Need Onions: E-graphs Modulo Theories via Bottom-up E-matching*. Apr. 19, 2025. DOI: 10.48550/arXiv.2504.14340. arXiv: 2504.14340[cs]. URL: http://arxiv.org/abs/2504.14340.